

E-Commerce Documentary Lab Codebase Summary

Internal inventory

Backend entry points

- 'backend/src/server.js' starts the Express app, registers middleware, mounts the API router at '/api', and listens on 'process.env.PORT || 5033'.

Frontend entry points

- 'frontend/src/main.jsx' bootstraps React with 'ReactDOM.createRoot(...)' and renders '<App />' inside 'React.StrictMode'.
- 'frontend/src/App.jsx' is the primary application shell and currently contains the UI, state, fetch logic, and documentation copy in one file.

Route definitions

- 'backend/src/routes/products.js'
 - 'GET /api/products'
 - 'GET /api/products/:id'
 - 'GET /api/collections'

API client definitions

- There is no dedicated API client module.
- 'frontend/src/App.jsx' issues 'fetch()' calls directly to '\${API_BASE}/products' and '\${API_BASE}/collections'.

Main state management files

- 'frontend/src/App.jsx'
 - 'products'
 - 'collections'
 - 'category'
 - 'query'
 - 'status'
- There is no external state library such as Redux, Zustand, or Context in the current repo.

Main config and env files

- 'backend/package.json' defines 'start' and 'dev' scripts and sets 'src/server.js' as the backend main file.
- 'frontend/package.json' defines 'dev', 'build', and 'preview' Vite scripts.
- 'frontend/src/App.jsx' reads 'import.meta.env.VITE_API_URL' and falls back to 'http://localhost:5033/api'.
- No '.env' file is present in the inspected repo.

1. High-level project overview

What this project is

- Fact: This lab is a small full-stack training repo with an Express backend and a React frontend. The README describes it as an "E-Commerce Documentary Lab" for practicing codebase reading and documentation generation.
- Fact: The backend serves a small in-memory product catalog from 'backend/src/data/catalog.json'.
- Fact: The frontend is a single-page dashboard that loads products and collections, offers category and keyword filters, and displays prompt-writing guidance for Codex.

Main folders

- 'backend/': Express API for catalog and collection data.
- 'frontend/': Vite + React UI that fetches the API and renders the training dashboard.
- 'docs/': Prompting guidance and documentary scaffolding for learners.

What a new engineer should know first

- Fact: The codebase is intentionally compact and easy to trace end to end.
- Fact: The primary runtime flow is 'App.jsx -> fetch /api/... -> Express router -> controller -> inventoryService -> catalog.json -> JSON response -> React state'.
- Inference: The lab is designed less as a production storefront and more as a teaching artifact for codebase understanding and documentation habits.

2. Backend explanation

Folder-by-folder

- 'backend/package.json'
 - Fact: Declares 'express' and 'cors' dependencies plus 'start' and 'dev' scripts.
- 'backend/src/server.js'
 - Fact: Creates the Express app, applies middleware, mounts the router, handles unknown routes, and starts the HTTP listener.
- 'backend/src/routes/products.js'
 - Fact: Central route table for product and collection endpoints.
- 'backend/src/controllers/productsController.js'
 - Fact: Thin controller layer that translates request inputs into service calls and HTTP responses.
- 'backend/src/services/inventoryService.js'
 - Fact: Core data-access and filtering logic for catalog reads.
- 'backend/src/data/catalog.json'
 - Fact: Static data store with four products across 'Apparel' and 'Footwear'.

Startup flow

1. 'backend/package.json' runs 'node src/server.js' for 'npm start'.
2. 'server.js' imports 'express', 'cors', and './routes/products'.
3. 'server.js' creates 'app', enables 'cors()' and 'express.json()', then mounts the

router at '/api'.

4. Requests that do not match the mounted API endpoints fall through to the final 404 JSON handler.

5. The app listens on port '5033' unless 'PORT' overrides it.

Middleware

- Fact: 'cors()' is globally enabled in 'backend/src/server.js'.
- Fact: 'express.json()' is globally enabled, even though the current routes are read-only 'GET' handlers.
- Fact: The final middleware is a catch-all 404 responder returning JSON with an error message.
- Inference: 'express.json()' is likely included as a default setup for future write endpoints, since the current API does not consume request bodies.

Routes, controllers, services, models

Routes

- 'GET /api/products'
 - Defined in 'backend/src/routes/products.js'.
 - Calls 'controller.listProducts'.
- 'GET /api/products/:id'
 - Defined in 'backend/src/routes/products.js'.
 - Calls 'controller.getProductById'.
- 'GET /api/collections'
 - Defined in 'backend/src/routes/products.js'.
 - Calls 'controller.listCollections'.

Controllers

- 'listProducts(req, res)'
 - Fact: Passes 'req.query' directly into 'inventoryService.listProducts'.
 - Fact: Wraps the response as '{ count, results }'.
- 'getProductById(req, res)'
 - Fact: Reads 'req.params.id'.
 - Fact: Returns '404' with '{ error: 'Product not found in catalog.' }' when the ID is missing.
- 'listCollections(req, res)'
 - Fact: Returns the array from 'inventoryService.listCollections()' as-is.

Services

- 'listProducts(filters = {})'
- Fact: Starts from 'catalog.products'.
- Fact: Filters by exact category match, case-insensitive.
- Fact: Filters by search term using 'product.name.toLowerCase().includes(term)' or 'p.tags.some(...)'.
- Fact: Returns all products matching criteria.

- Fact: Returns a reduced shape with only 'id', 'name', 'price', 'category', and 'inventory'.
- Inference: The service intentionally hides fields like 'description', 'type', and 'tags' from list responses to keep list payloads smaller.
- 'getProductById(id)'
 - Fact: Returns the first exact ID match from 'catalog.products'.
 - Fact: Returns the full product object, including description and tags, when found.
- 'listCollections()'
 - Fact: Builds two hard-coded collection groups by filtering category values 'Apparel' and 'Footwear'.
 - Fact: Returns only 'name' and nested item '{ id, name }' pairs for each collection.

Request lifecycle

1. Browser requests '/api/products', '/api/products/:id', or '/api/collections'.
2. 'server.js' routes the request into 'productsRouter' through the '/api' mount.
3. 'productsRouter' selects the controller function.
4. The controller calls 'inventoryService'.
5. 'inventoryService' reads from 'catalog.json', optionally filters or reshapes data, and returns plain JavaScript objects.
6. The controller serializes the service result with 'res.json(...)'.
 - 7. If no route matches, the 404 middleware returns a JSON error payload.

Dependency flow

- 'server.js' depends on 'routes/products.js'.
- 'routes/products.js' depends on 'controllers/productsController.js'.
- 'productsController.js' depends on 'services/inventoryService.js'.
- 'inventoryService.js' depends on 'data/catalog.json'.
- Inference: This is a deliberate "router -> controller -> service -> data" teaching architecture rather than a necessity for a four-item catalog.

Files to read first

- 'backend/src/server.js': establishes the runtime shell and the real port.
- 'backend/src/routes/products.js': fastest way to see the API surface.
- 'backend/src/controllers/productsController.js': clarifies response shapes and error behavior.
- 'backend/src/services/inventoryService.js': contains the only meaningful backend business logic.
- 'backend/src/data/catalog.json': shows the source data fields and sample inventory.

Risks

- Fact: There is no validation layer for query params or route params.
- Fact: There is no persistence beyond the JSON file, so all data is static.
- Fact: Search is partially case-insensitive: product names are lowercased before

matching, but tags are not lowercased before 'includes(term)'.

- Inference: A query such as uppercase tag input could produce inconsistent results if tag casing changes in future data.
- Fact: The router and service are read-only, so the included 'express.json()' middleware is unused today.

3. Frontend explanation

Folder-by-folder

- 'frontend/package.json'
 - Fact: Uses Vite, React, and '@vitejs/plugin-react-swc'.
- 'frontend/index.html'
 - Fact: Provides the root HTML shell for Vite.
- 'frontend/src/main.jsx'
 - Fact: Imports 'App' and 'styles.css', then mounts React.
- 'frontend/src/App.jsx'
 - Fact: Holds all current UI structure, API requests, state, and prompt examples.
- 'frontend/src/styles.css'
 - Fact: Contains the app styling for the page, panels, grid, cards, and documentation sections.

Bootstrap flow

1. Vite serves 'frontend/index.html'.
2. 'frontend/src/main.jsx' loads and mounts '<App />'.
3. 'App.jsx' initializes state for products, collections, filters, and loading status.
4. The first 'useEffect' loads products, and re-runs when 'category' or 'query' changes.
5. The second 'useEffect' loads collections once on mount.
6. 'useMemo' computes aggregate counts from the current 'products' array.

Routing

- Fact: There is no client-side router such as React Router.
- Fact: The app is a single-screen dashboard rendered entirely by 'App.jsx'.
- Inference: If the lab grows, a product detail view would likely be the first feature to justify introducing routing because the backend already exposes 'GET /api/products/:id'.

Pages, components, hooks, state, API layer

- Pages/components
 - Fact: There is only one top-level component, 'App'.
 - Fact: The product cards, filters, collections section, and prompt log are rendered inline rather than split into component files.
- Hooks
 - Fact: 'useState' manages local UI state.
 - Fact: Two 'useEffect' hooks handle product and collection loading.

- Fact: 'useMemo' computes '{ total, inventory }' from loaded products.
- State
 - Fact: 'products' drives the catalog grid.
 - Fact: 'collections' drives the documentary highlights list.
 - Fact: 'category' and 'query' drive the product-fetch URL.
 - Fact: 'status' communicates loading and failure states in the UI.
- API layer
 - Fact: There is no shared API module or custom hook.
 - Fact: 'fetch()' is called inline in 'App.jsx'.
 - Fact: 'API_BASE' defaults to 'http://localhost:5033/api' and can be overridden with 'VITE_API_URL'.

Render and data flow

1. User lands on the page and sees the hero plus empty/loading panels.
2. Product effect builds 'URLSearchParams' from 'category' and 'query'.
3. Frontend requests '\${API_BASE}/products?...'
4. Successful responses populate 'products' and update 'status'.
5. The grid maps 'products' into cards, and 'inventorySummary' recalculates totals.
6. A second request to '\${API_BASE}/collections' populates the collections panel.
7. Static prompt-copy content is rendered from the 'samplePrompts' object and hard-coded explanatory paragraphs.

Component hierarchy

```
main.jsx
-> App
  -> header.hero
  -> section.panel (catalog snapshot)
    -> filters
    -> grid
      -> article.card x N
    -> summary
  -> section.panel (collections)
    -> collection x N
      -> list item x N
  -> section.panel (prompt strategy log)
    -> function-level article
    -> folder-level article
    -> documentary note
```

Files to read first

- 'frontend/src/main.jsx': confirms the real frontend entry point.
- 'frontend/src/App.jsx': contains the app's runtime behavior and is the highest-value file for onboarding.
- 'frontend/src/styles.css': useful after 'App.jsx' if you need to map class names to

layout or visual intent.

- 'frontend/package.json': confirms the toolchain and available scripts.

Risks

- Fact: 'App.jsx' mixes presentation, data fetching, state management, and training copy in one file.
- Fact: The catalog fetch handles non-OK responses but does not catch network errors, so a thrown fetch failure would skip the custom status update.
- Fact: The collections fetch swallows errors with `'.catch(() => {})'`, which makes failures invisible to the UI.
- Fact: The prompt examples mention 'frontend/src/components', a 'ProductGrid', and a CTA bar, but those files or abstractions do not exist in the inspected repo.
- Fact: Several UI strings display encoding artifacts such as 'A^'-style mojibake, suggesting text-encoding issues in source content.

4. Frontend-backend flow

API definitions

- 'GET /api/products'
 - Query params: optional 'category', optional 'q'
 - Response shape: '{ count: number, results: Array<{ id, name, price, category, inventory }> }'
- 'GET /api/products/:id'
 - Response shape: full product object or '404'
- 'GET /api/collections'
 - Response shape: '[[name, items: [{ id, name }]]'

Backend route mapping

- 'frontend/src/App.jsx' product loader -> 'backend/src/routes/products.js' -> 'productsController.listProducts' -> 'inventoryService.listProducts'
- 'frontend/src/App.jsx' collections loader -> 'backend/src/routes/products.js' -> 'productsController.listCollections' -> 'inventoryService.listCollections'
- Fact: No current frontend code calls 'GET /api/products/:id'.

Response handling

- Products
 - Fact: The frontend expects 'payload.results' and 'payload.count'.
 - Fact: The grid and summary derive entirely from 'products'.
- Collections
 - Fact: The frontend expects an array and passes it directly into 'setCollections'.
 - Inference: Because there is no runtime validation, backend response-shape changes would break the page silently or partially.

Concrete end-to-end journeys

Journey 1: Initial catalog load

1. 'App.jsx' mounts.
2. The product 'useEffect' runs because 'category' and 'query' are initialized.
3. The frontend fetches '/api/products' with no filters.
4. 'inventoryService.listProducts({})' returns all four catalog items in reduced list form.
5. The frontend stores the result and renders four product cards plus aggregate totals.

Journey 2: Category filter

1. User selects 'Apparel' or 'Footwear' in the '<select>'.
2. 'setCategory(...)' triggers a re-render and re-runs the product 'useEffect'.
3. The frontend requests '/api/products?category=Apparel' or '/api/products?category=Footwear'.
4. 'inventoryService.listProducts' filters by exact category match after lowercasing both values.
5. The frontend updates the grid and recalculates total visible inventory.

Journey 3: Keyword search

1. User types into the keyword input.
2. 'setQuery(...)' re-runs the product effect.
3. The frontend requests '/api/products?q=<term>'.
4. The service matches against product names or tags and returns only matching items.
5. The status text updates to show the new result count.

Journey 4: Collections panel load

1. On first mount, the second 'useEffect' requests '/api/collections'.
2. The backend groups products into 'Apparel Highlights' and 'Footwear Spotlight'.
3. The frontend renders each collection name and nested item list in the second panel.

5. Execution flow

Frontend boot

```
index.html
-> src/main.jsx
-> App.jsx
  -> useEffect(fetch products)
  -> useEffect(fetch collections)
  -> render hero, filters, cards, collections, prompt log
```

Products request

```
App.jsx fetch(/api/products?... )
-> server.js mounts /api
-> routes/products.js
  -> productsController.listProducts
    -> inventoryService.listProducts
```



```
-> data/catalog.json
-> res.json({ count, results })
-> App.jsx setProducts + setStatus
```

Collections request

```
App.jsx fetch(/api/collections)
-> routes/products.js
-> productsController.listCollections
-> inventoryService.listCollections
-> data/catalog.json
-> res.json(collections)
-> App.jsx setCollections
```

6. Architecture summary

- Fact: The backend follows a classic layered Express shape: router, controller, service, static data.
- Fact: The frontend uses a single React component instead of a decomposed component tree.
- Fact: API communication is direct and simple, with no adapter or repository layer.
- Inference: The architecture is optimized for teachability and traceability rather than scalability.
- Inference: The clearest future refactor would be splitting 'App.jsx' into 'CatalogPanel', 'CollectionsPanel', and 'PromptStrategyPanel', then moving 'fetch' logic into a small API module or custom hooks.

7. Onboarding guide

Fastest way to get productive

1. Read 'backend/src/server.js' to confirm the actual API port and mounted base path.
2. Read 'backend/src/routes/products.js' and 'backend/src/controllers/productsController.js' together to understand the HTTP surface.
3. Read 'backend/src/services/inventoryService.js' and 'backend/src/data/catalog.json' to understand the data model and response shaping.
4. Read 'frontend/src/main.jsx' and 'frontend/src/App.jsx' to trace how the UI talks to the backend.
5. Read 'docs/Documentation-prompt.md', 'docs/Documentary-template.md', and 'docs/prompt-strategies.md' to understand the instructional goal of the lab.

Good first changes

- Extract product fetching into a small 'frontend/src/api.js'.
- Split the large 'App.jsx' file into reusable components.
- Add explicit error UI for collections loading.
- Add a frontend path that uses 'GET /api/products/:id', since that endpoint currently has no UI consumer.

- Fix README drift so it matches the codebase's real port and folder structure.

8. Text diagrams

Backend dependency diagram

```
server.js
-> routes/products.js
    -> controllers/productsController.js
        -> services/inventoryService.js
            -> data/catalog.json
```

Frontend state and render diagram

```
category/query input
-> useState updates
    -> products useEffect reruns
        -> fetch /api/products
            -> setProducts
        -> useMemo inventorySummary
            -> grid + totals render
```

```
mount
-> collections useEffect
    -> fetch /api/collections
        -> setCollections
            -> collections panel render
```

Lab-purpose diagram

```
Codebase
-> runtime code (backend + frontend)
-> docs scaffolding (prompt + template + strategies)
    -> learner asks Codex for summaries
        -> learner verifies statements against source
            -> learner produces documentary-style documentation
```

9. Codebase risks and improvements

Evidence-based risks

- Fact: 'README.md' says the backend listens on port '4000', but 'backend/src/server.js' uses '5033' by default.
- Fact: 'README.md' and 'Documentary-template.md' reference components such as 'frontend/src/components', 'ProductGrid', and a CTA panel, but those do not exist in the current source tree.
- Fact: The frontend ignores or hides some failures, especially in the collections request.
- Fact: The service logic has no tests, even though it contains the main filtering

behavior.

- Fact: Search normalization is inconsistent between names and tags.

Suggested improvements

- Update 'README.md' and template text so training instructions match the current repo.
- Add lightweight tests for 'inventoryService.listProducts', 'getProductById', and 'listCollections'.
- Normalize tags with 'tag.toLowerCase().includes(term)' for consistent search behavior.
- Extract frontend UI sections into real components so the documentary prompts can reference actual files.
- Add a product detail panel or route to use the existing 'GET /api/products/:id' endpoint.

Fact vs inference note

- Fact labels above come directly from inspected files in 'backend/', 'frontend/', 'docs/', and the root 'README.md'.
- Inference labels mark likely intent, probable future direction, or design rationale that is not explicitly stated in executable code.